REAL TIME SLIDING-WINDOW LOG ANALYTICS
AND
ANOMALY DETECTION ENGINE

Submitted by

Abhishek Sharma

Registration No- 12527924

Roll No- R9P163A61

Section – 9P163

Under the Guidance of

Mr. Ashish Kumar Pandey

UID: 66753

Lovely School of Computer Science and Engineering

Lovely Professional University, Phagwara

CERTIFICATE

This is to certify that Abhishek Sharma bearing Registration no. 12527924 has completed project titled "Real-Time-Sliding-Window-Log-Analytics-and-Anomaly-Detection-Engine", under my guidance and supervision. To the best of my knowledge, the present work is the result of his/her original development, effort and study.

Signature and Name of the Supervisor

Designation of the Supervisor

School of Computer Science and Engineering
Lovely Professional University
Phagwara, Punjab.

Date:

DECLARATION

I, Abhishek Sharma student of M.Tech Data Science and Analytics under CSE/IT Discipline at, Lovely Professional University, Punjab, hereby declare that all the information furnished in this project report is based on my own intensive work and is genuine.

Date: Signature

Registration No.12527924 Name of the student

Abhishek Sharma

Table of Content

1. Introduction
2. Scope of Project
3. Existing System
4. System Workflow
5. Core Data Structures and Algorithms
6. Implementation Details
7. Visualization and Monitoring Interface
8. Future scope
9. References

Introduction

Modern distributed applications continuously generate large volumes of log data from services such as authentication, payments, search, and inventory systems. These logs provide critical insights into system behavior, traffic patterns, and error conditions. Delayed analysis of such data can result in undetected failures, degraded performance, or security risks. As a result, real-time log analytics has become essential for effective system monitoring and anomaly detection.

Traditional batch-processing methods introduce latency between log generation and analysis. In contrast, streaming-based approaches process events as they arrive, enabling near real-time metric computation within bounded time windows. This allows systems to detect abnormal patterns and respond more quickly to operational issues.

This project presents the design and implementation of a real-time sliding window log analytics and anomaly detection engine using FastAPI as the service layer. The system continuously ingests log events, maintains a 60-second sliding window of recent logs, and performs stateful aggregation using efficient data structures such as deques and hash maps. A heap-based Top-K mechanism identifies heavy-hitter IP addresses, while rule-based thresholds detect abnormal error rates and suspicious traffic spikes.

The architecture emphasizes low-latency processing, constant-time updates per event, and bounded memory usage. Metrics are exposed through RESTful endpoints implemented with FastAPI and visualized through a live dashboard. Although implemented as a single-node in-memory system, the design reflects core principles of real-time streaming analytics used in larger-scale systems.

Scope of the Project

The scope of this project is focused on designing and implementing a real-time, in-memory log analytics engine capable of processing streaming log events within a bounded time window. The system is intended to demonstrate core streaming concepts such as sliding window processing, stateful aggregation, heavy-hitter detection, and rule-based anomaly evaluation.

The project includes:

- Continuous ingestion of log events through a simulated streaming mechanism
- Maintenance of a time-based sliding window (60 seconds)
- Real-time aggregation of service-level and IP-level metrics
- Computation of error rates within the active window
- Top-K heavy-hitter detection using a heap-based approach
- Threshold-based anomaly detection for error spikes and suspicious IP activity
- Exposure of metrics through FastAPI-based RESTful endpoints
- Visualization of live system metrics through a dashboard interface

The system operates as a single-node, in-memory application designed for low-latency processing and bounded memory usage. It does not include distributed scaling, persistent storage, fault tolerance mechanisms, or advanced machine learning-based anomaly detection.

While limited to a prototype-level implementation, the project scope covers fundamental architectural principles used in real-time log processing systems and provides a foundation for extending the system toward distributed and production-grade environments.

Existing System

Existing log management approaches can be categorized as follows:

1. Traditional Batch Processing Systems

- Logs are stored over time and analyzed periodically.
- Processing occurs at fixed intervals (hourly, daily, etc.).
- High latency between log generation and analysis.
- Not suitable for immediate anomaly detection.

2. Basic Monitoring Setups

- Use static dashboards and scheduled queries.
- Provide historical insights rather than real-time analytics.
- Limited support for sliding window computation.
- No dynamic heavy-hitter or live anomaly detection.

3. Enterprise Log Management Platforms (e.g., ELK Stack, Splunk)

- Support real-time analytics and advanced querying.
- Capable of distributed processing and scaling.
- Require significant infrastructure and configuration.
- High operational and resource overhead.

System Workflow

The system operates as a continuous real-time streaming pipeline. Each log event passes through multiple stages, where it is processed, aggregated, evaluated, and exposed through APIs for visualization.
The overall workflow is described below:

1. Log Ingestion

- A background thread continuously generates log events.
- Each log contains:
  - Timestamp
  - Service name
  - Log level (INFO, WARN, ERROR)
  - IP address
- Logs are immediately forwarded to the processing engine.

This simulates a real-time event stream.

2. Sliding Window Update

- Incoming logs are appended to a deque.
- The system maintains a time-based window of 60 seconds.
- When a new log arrives:
  - It is added to the window.
  - Logs older than 60 seconds are evicted.

This ensures bounded memory usage and time-constrained analytics.

3. Real-Time Aggregation

For every log event:

- Error count is updated if the log level is ERROR.
- Service-level request count is incremented.
- IP-level request frequency is updated.

During eviction:

- Corresponding counts are decremented.
- Keys with zero frequency are removed.

This maintains accurate, real-time state within the active window.

4. Heavy-Hitter (Top-K) Computation

- On request, the system computes the top K IP addresses.
- A min-heap is constructed from current IP frequencies.
- The heap retains only the most active IPs.

This enables detection of concentrated traffic sources.

5. Anomaly Detection

The system evaluates rule-based conditions:

- High error rate:
    - Triggered if error_count / total_logs exceeds threshold.
- Suspicious IP traffic:
    - Triggered if any IP exceeds configured request limit.

If conditions are met, alerts are generated and stored.

6. API Exposure and Visualization

- Metrics are exposed through FastAPI endpoints:
    - Error rate
    - Top IPs
    - Service load
    - Alerts
- The dashboard polls these endpoints every 2 seconds.
- Charts and tables update dynamically.

This enables live system monitoring.

.

Core Data Structures and Algorithms

The system relies on efficient in-memory data structures to support real-time processing with low computational overhead. Each structure was selected to ensure constant-time updates and bounded memory usage within the sliding window.

1. Deque (Sliding Window Management)

A double-ended queue (deque) is used to maintain the time-based sliding window.

Purpose:

- Store logs within the last 60 seconds.
- Support fast insertion and removal operations.

Why Deque:

- O(1) append operation for incoming logs.
- O(1) removal from the front during eviction.

Efficient for FIFO-based time window logic.

Role in System:

- Ensures only recent logs are considered.
- Guarantees bounded memory usage.
- Maintains consistency for real-time analytics.

2. Hash Maps (Stateful Aggregation)

Hash maps are used for real-time metric tracking.

Maintained Maps:

- Service-level request counts.
- IP-level request frequencies.

Why Hash Map:

- Average O(1) insertion and lookup.
- Efficient frequency counting.
- Dynamic key addition and removal.

Role in System:

- Enables fast metric updates per log event.
- Supports accurate sliding window aggregation.
- Provides input for anomaly detection and Top-K computation.

3. Min-Heap (Top-K Heavy-Hitter Detection)

A min-heap of size K is used to compute the most active IP addresses.

Purpose:

- Identify heavy traffic sources within the window.
- Track top K IPs efficiently.

Why Min-Heap:

- Maintains smallest element at root.
- Ensures heap size remains bounded to K.
- Efficient Top-K selection with O(n log k) complexity.

Role in System:

- Detects traffic concentration.
- Supports suspicious activity analysis.

4. Scalar Counters (Error Tracking)

A simple integer counter tracks the number of ERROR-level logs.

Purpose:

- Compute error rate within the sliding window.
- Enable quick anomaly checks.

Complexity:

- O(1) increment and decrement.
- O(1) error rate computation.

Algorithmic Complexity Summary

Operation	Complexity
Log insertion	O(1)
Log eviction	O(1) amortized
Aggregation update	O(1)
Error rate calculation	O(1)
Top-K computation	O(n log k)

Where:

- n = number of unique IPs in window
- k = configured Top-K value

These data structures collectively enable low-latency, stateful streaming analytics while maintaining computational efficiency and bounded memory usage.

Implementation details

The system is implemented as a single-node, in-memory streaming application using Python and FastAPI. The architecture separates ingestion, processing, and API exposure into modular components to maintain clarity and extensibility.

1. Log Generation Module

- A synthetic log generator simulates real-time event streams.
- Each log includes:
  - Timestamp
  - Service name
  - Log level (INFO, WARN, ERROR)
  - IP address
- A background thread continuously generates logs and forwards them to the processor.

This simulates streaming ingestion without requiring external message brokers.

2. Processing Engine (LogProcessor Class)

The core processing logic is encapsulated within a LogProcessor class.

Responsibilities:

- Maintain sliding window using a deque.
- Update error counters and aggregation maps.
- Evict logs older than 60 seconds.
- Compute Top-K IPs on demand.
- Evaluate anomaly rules and generate alerts.

All state is maintained in memory to ensure fast read and write operations.

3. Configuration Management

System parameters are defined in a configuration module:

- Window duration (60 seconds)
- Top-K value
- Error rate threshold
- IP request threshold

This enables easy tuning without modifying processing logic.

4. FastAPI Service Layer

FastAPI is used to expose metrics through RESTful endpoints.

Available endpoints:

- /metrics/error_rate
- /metrics/top_ips
- /metrics/service_load
- /alerts

The API layer interacts directly with the processor instance and returns real-time metrics.

5. Dashboard Interface

A lightweight HTML dashboard is served through FastAPI.

Features:

- Line chart for error rate trend
- Bar chart for service-level load
- Table for Top-K IPs
- Alert display banner

The dashboard polls API endpoints every 2 seconds to update visualizations dynamically.

6. Execution Flow

1. Application starts.
2. Background thread begins generating logs.
3. Logs are processed immediately.
4. Metrics are continuously updated.
5. API endpoints expose live results.
6. Dashboard reflects updated metrics in near real time.

The implementation demonstrates stateful streaming analytics with low latency and bounded memory usage within a single-node architecture.

Visualization and Monitoring Interface

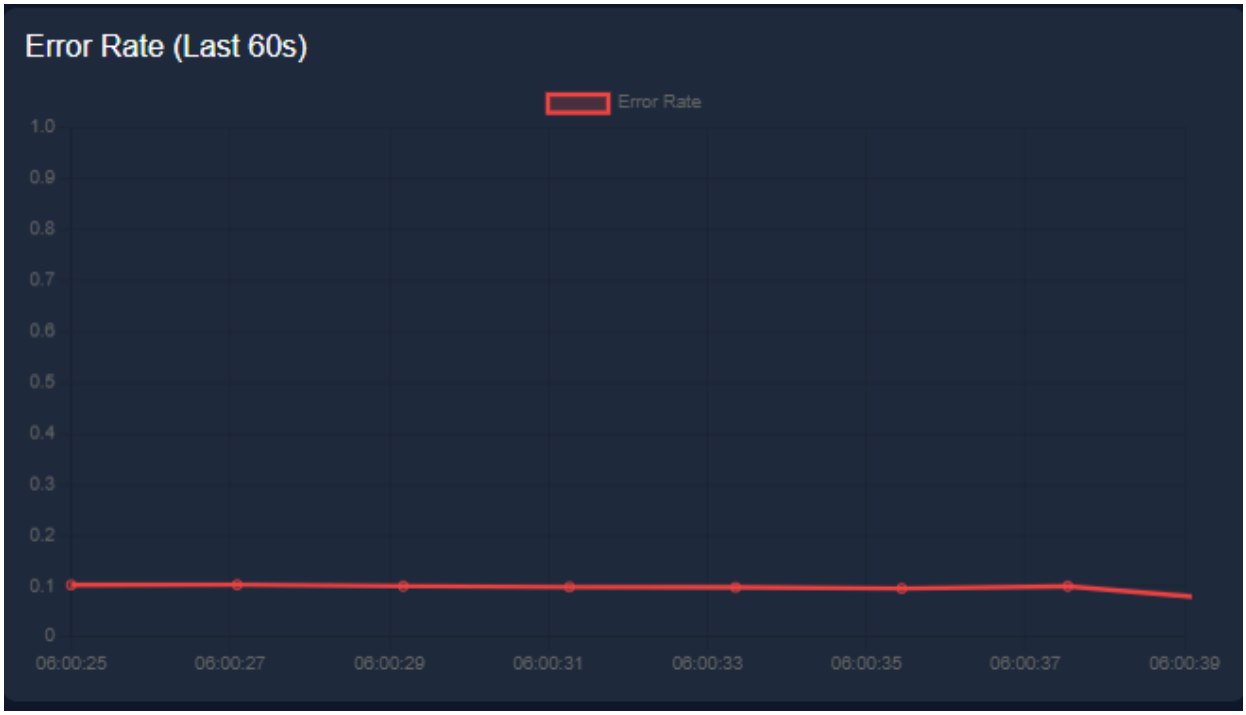
The system includes a real-time dashboard designed to provide live visibility into system behaviour and traffic patterns. The dashboard retrieves data from FastAPI endpoints at fixed intervals and updates visual components dynamically.

1. Error Rate Trend (Line Chart)

- Displays error rate over time (last 60-second window).
- Updated every 2 seconds.
- Helps identify sudden spikes in failures.
- Supports early detection of service instability.

Purpose:

Provides temporal visibility into system reliability.

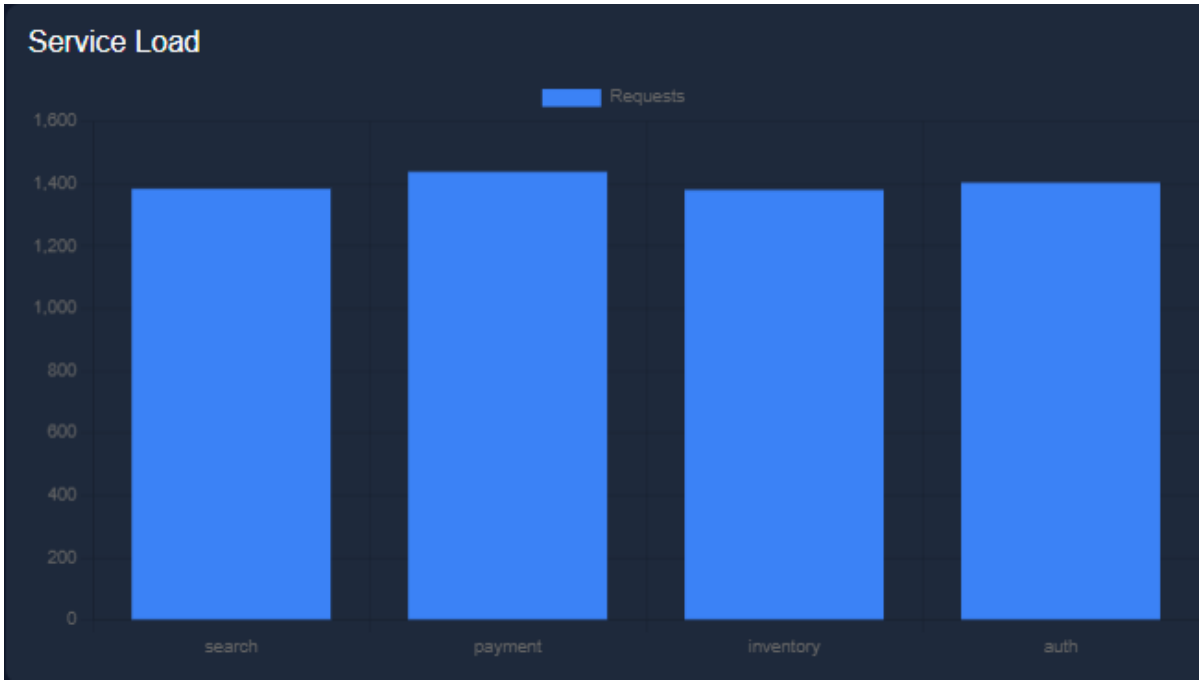


2. Service Load Distribution (Bar Chart)

- Displays request count per service (auth, payment, search, inventory).
- Reflects current load within sliding window.
- Helps identify uneven traffic distribution.

Purpose:

Supports workload monitoring and service-level analysis.



3. Top-K IP Table

- Displays most active IP addresses.
- Shows request count per IP.
- Updated dynamically from heap-based computation.

Purpose:

Identifies traffic concentration and potential suspicious behavior.

Top IPs

IP Address	Requests
192.168.0.99	1678
192.168.0.95	25
192.168.0.220	25

Visualization Characteristics

- Real-time updates via periodic API polling.
- Lightweight front-end using Chart.js.
- No data persistence; reflects current sliding window state only.
- Designed for monitoring rather than historical analytics.

Future Scope

Although the current system demonstrates real-time sliding window analytics in a single-node environment, several enhancements can significantly improve scalability, robustness, and analytical capability.

1. Distributed Log Ingestion

- Integrate a message broker such as Kafka or Redis Streams.
- Enable scalable, fault-tolerant event ingestion.
- Support higher throughput and multiple producers.

2. Persistent Storage Layer

- Store logs in a database (e.g., PostgreSQL, ClickHouse).
- Enable historical analysis and replay capability.
- Support long-term trend analysis beyond the sliding window.

3. Distributed Stream Processing

- Extend architecture using frameworks such as Apache Flink or Spark Streaming.
- Implement partitioned processing for horizontal scalability.
- Introduce checkpointing and fault tolerance.

4. Advanced Anomaly Detection

- Replace static thresholds with statistical models.
- Implement techniques such as:
    - Moving averages (EWMA)
    - Z-score based detection
    - Adaptive baselines per service or IP
- Improve accuracy and reduce false positives.

5. Performance Optimization

- Incremental Top-K maintenance instead of rebuilding heap on demand.
- Introduce concurrency controls for thread safety.
- Optimize memory usage under high-cardinality workloads.

6. Enhanced Monitoring and Observability

- Integrate Prometheus for metrics collection.
- Add latency tracking and throughput monitoring.
- Implement structured logging and tracing.

7. Security and Access Control

- Add authentication and role-based access to API endpoints.
- Secure endpoints for production deployment.

These enhancements would transform the current prototype into a scalable, production-ready real-time log analytics platform capable of handling large-scale distributed environments.

References

1. Python Software Foundation. (2024). *Python 3 Documentation – Standard Library*. https://docs.python.org/3/library/
2. Python Software Foundation. (2024). *Python 3 Documentation – Standard Library*. https://docs.python.org/3/library/
3. Python Software Foundation. (2024). *heapq — Heap queue algorithm*. https://docs.python.org/3/library/heapq.html
4. FastAPI Documentation. (2024). *FastAPI Official Documentation* . https://fastapi.tiangolo.com/
5. Akidau, T., Chernyak, S., & Lax, R. (2018). *Streaming Systems: The What, Where, When, and How of Large-Scale Data Processing*. O'Reilly Media.